Formatted I/O: scanf

CSC 100 Introduction to programming in C/C++, Spring 2025

Table of Contents

1. README

- There is much more to `scanf` and `printf` than we've seen
- I/O is where the pedal hits the metal - where man meets machine
- In this notebook: conversion specifications for `scanf`
- Practice workbooks, input files and PDF solution files in GitHub

2. The secret of the three doors

There are three doors that lead in and out of your computer.

These are the only doors that data can get through:

1. `stdin` or "Standard input" - for example from the keyboard
2. `stdout` or "Standard output" - for example the screen
3. `stderr` or "Standard error" - all errors and warnings

If you want to read from, or write to files instead the standard input/output devices, you need to build and install new doors.

3. scanf

- A `scanf` **format string** may contain ordinary characters and conversion specifications like `d`, `e`, `f`, `g`
- The **conversions** allowed with `scanf` are essentially the same as those used with `printf`
- The `scanf` format string tends to contain **only** conversion specs

4. First example

- Open a OneCompiler.com file and code along.
- Example input:

```
1  -20  .3   -4.0e3
```

- Emacs: Create input file

```
echo "1  -20 .3 -4.0e+3" > input  # store string in file `input`
cat input  # view the file `input`
```

- Example program to read this input:

```
int i, j;
float x, y;

scanf("%d%d%f%e", &i, &j, &x, &y);

printf("|%5d|%5d|%5.1f|%10.1e|\n", i, j, x, y);
```

```
|    1|    5|-100.0|   0.0e+00|
```

5. Main traps when using `scanf`

- The compiler will not check that specs and variable input match up.
- The `&` pointer symbol must not miss in front of the input variable.
- `scanf` works in mysterious ways (we'll see why in a moment)

6. How scanf works

- `scanf` is a pattern-matching function: it tries to match input groups with conversion specifications in the format string

- For each spec, it tries to locate an item in input
- It reads the item, and stops when it can't match
- If an item is not read successfully, `scanf` aborts

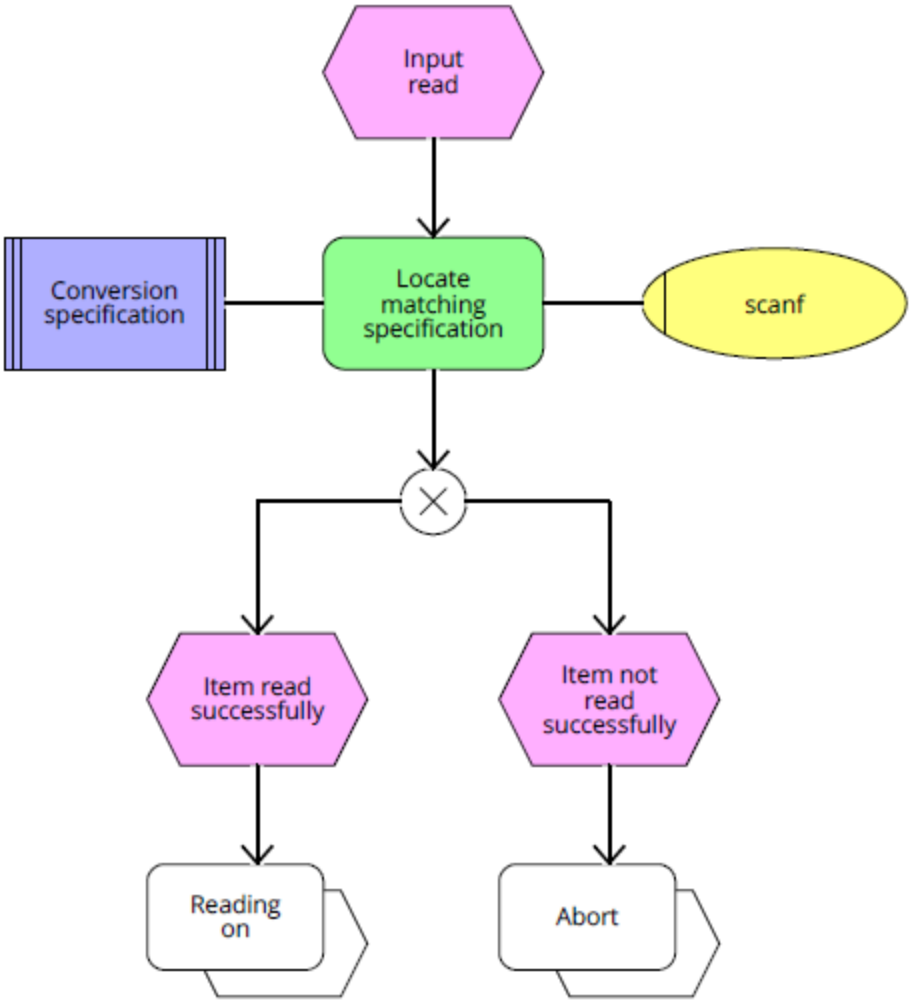

Figure 1: How scanf works (Event-controlled Process Chain diagram)

- Ignores white-space: space (`" "`), TAB (`\t`), new-line (`\n`)
- Input can be on one line or spread over several lines:

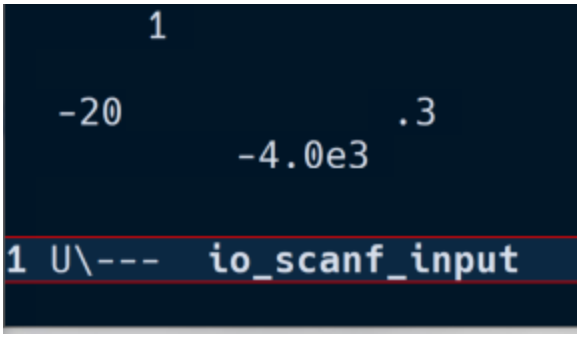

Figure 2: Input file for scanf

- **Try this in OneCompiler.com now!**
- `scanf` sees a character stream (¤ = new-line, `s` = skip'd, `r` = read):

```
••1¤-20•••.3¤•••-4.0e3¤   // format string: "%d%d%f%e"
ssrsrrrsssrrssssrrrrrr
```

  - When asked to read an **integer** (`%d` or `%i`), `scanf` searches for a digit, or a +/- sign, then reads until it encounters a non-digit
  - When asked to read a **float** (`%f`, `%g`, `%e`), `scanf` looks for +/- sign, digits, decimal point, or an exponent (`e+02`, `e-02`)
  - When used with `scanf`, `%e`, `%f`, `%g` are completely interchangeable (**try that in OneCompiler.com with the last format specifier**).
  - When it finds a character that cannot be part of the current item, the character is returned to be read again during the scanning of the next input item or the next call of `scanf`.

7. Walk through example

This example has the same spec as our earlier example: `"%d%d%f%f",&i,&j,&x,&y`. This is what the computer "sees":

```
1-20.3-4.0e3¤
```

1. Expects `%d`. Stores `1` in `i`, returns `-`
2. Expects `%d`. Stores `-20` in `j`, returns `.`
3. Expects `%f`. Stores `0.3` in `x`, returns `-`
4. Expects `%f`. Stores `-4.0 x 10^3` in `y`, returns ¤ and finishes.

8. Ordinary characters in format strings

- `scanf` reads white-space until it reaches a symbol.
- When it reaches a symbol, it tries to match to next input.
- It now either continues processing or aborts.

- Example: input contains "1. 3.56 100 5 .1" - how to scan?

```
float x=2., y=8., z; // initial values
int   i=10, j=20;

scanf("%f%f%d%d%f", &x, &y, &i, &j, &z);
printf("%.1f %.2f %d %d %.1f",  x,  y,  i,  j, z);
```

```
1.0 3.56 100 5 0.1
```

- To create the input file on the shell:

```
echo "1. 3.56 100 5 .1" > input2
cat ./input2
```

9. Example with ordinary characters

- If the format string is `"%d/%d"` and the input is `•5/•96`, `scanf` succeeds: once the `/` is scanned, any number of white spaces are ignored.
- If the input is `•5•/•96` , `scanf` fails, because the `/` in the format string doesn't match the space in the input: an `/` is expected immediately.

  > Details: After reading the first integer, `scanf` expects to find a `/` character immediately. It encounters a whitespace character instead, which is not skipped because the whitespace is not leading (from scanf's perspective at this point; it's looking for a specific non-whitespace character, `"/"`, and aborts.

- To allow spaces after the first number, use `"%d•/%d"` instead.

10. Common mistakes:

1. Putting `&` in front of variables in a `printf` call

   ```
   printf("%d %d\n", &i, &j);  /*** WRONG ***/
   ```

2. Assuming that `scanf` should resemble `printf` formats

   ```
   scanf("%d, %d", &i, &j);  // works only if input: `500, 400`
   ```

   - After storing `i`, `scanf` will try to match a comma with the next input character. If it's a space, it will abort.
   - For this example, only the input `100, 100` works, but not `100 100`

3. Putting a `\n` character at the end of `scanf` string

   ```
   scanf("%d\n", &i);
   ```

   - To `scanf`, the new-line is *white-space*. It will advance to the next white-space character and not finding one will hang forever

11. The golden rule

- To avoid trouble with `scanf`, simply avoid any ordinary characters in the format string if you can.
- If you need ordinary characters, make sure that the format string matches the input structure exactly.
- Example: I want to enter a phone number with three dashes:

  Input:

```
echo "800-123-5678" > input
cat input
```

```
800-123-5678
```

To scan it in this form, I need to copy the structure "X-Y-Z":

```
int i,j,k;
scanf("%i-%i-%i", &i, &j, &k);
printf("Phone number: (%i) %i-%i", i, j, k);
```

```
Phone number: (800) 123-5678
```

12. PRACTICE Reading input with `scanf`

- You can open the exercises here on GitHub whenever you're lost. tinyurl.com/scanf-practice-26
- We'll do these in class together and you will upload your results to Canvas (`"In-class practice 8: input with scanf"`).
- You will learn:
  - How to open a cloud command-line terminal (aka shell)
  - Create a new file with GNU `nano`
  - Save and rename files
  - Change and make directories
  - List, move, delete and rename files
  - Compile with changing the object file name
  - Create an input file with `echo`
  - Run an executable using a relative path
  - Open the editor and download a file
  - Compress files and upload archive files to Canvas

Author: Marcus Birkenkrahe

Created: 2026-03-03 Tue 22:41